SECJ 3483 - Web Technology
Week 02: Hands-On Lab Activities - HTML DOM

DOM Selection, Manipulation, and Event Handling.
5 lab activities, each building on the previous one.
Use the following HTML and gradually enhance it.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>DOM Demo: To-Do List</title>
  <style>
    body { font-family: Arial; margin: 20px; }
    .done { text-decoration: line-through; color: gray; }
    li { margin: 5px 0; }
  </style>
</head>

<body>
  <h2>To-Do List</h2>
  <input type="text" id="itemInput" placeholder="Enter new task">
  <button id="addBtn">Add</button>
  <ul id="todoList"></ul>

  <!-- Floating span -->
  <span id="focusBtn" class="focus-button">focus on me</span>

  <script>
      //Activity 1-5: write code here
  </script>
</body>
</html>
```

Activity 1: Display Alert on Button Click

Objective: Learn basic DOM selection and event handling.

Goal: Add a button that displays an alert when clicked.

What you need to do:

1. Write JavaScript code to select the **button** element.
2. Attach an event listener to the button that triggers an alert when clicked.
3. The alert should display the message: **"Hello DOM!"**.

Javascript Code:

_____
_____

JQuery Code:

_____
_____

.

Activity 2: Display Input Value in Alert

Objective: Read input values from the textbox and show the value in an alert message.

Goal: Display the value entered in an input box in an alert when the button is clicked.

What you need to do:

1. Select the input and button elements.
2. Attach an event listener to the button that reads the value of the input field.
3. Display the entered value in an alert box with the format: "You typed: [input value]".

Javascript Code:

_____
_____

JQuery Code:

_____
_____

Activity 3: Add Items to a List

Objective: Dynamically create elements and add them to the page.

Goal: Allow the user to add items to a list using an input field.

What you need to do:

1. Write JavaScript code to capture the input value when the **"Add"** button is clicked.

2. Dynamically create a new `<li>` element using the input value and append it to the `<ul>` with `id="todoList"`.

3. Clear the input field after the item is added.

Javascript Code:

_____
_____

JQuery Code:

_____
_____

Activity 4: Add Interactivity to List Items

Objective: Add multiple event listeners to elements.

Goal: Enable interactivity in the list items: toggle a "done" class on click (strikethrough) and remove the item on double-click.

What you ~~need~~ to do:

1. Write ~~code~~ to toggle the `.done` class on the `<li>` when clicked, which will apply a strikethrough effect.

2. Add an event listener to remove the `<li>` element from the list when it is double-clicked.

3. Use CSS to define the `.done` class, ensuring it applies a line-through style to the text.

Javascript Code:

```
//activity 4
  document.getElementById('addBtn').addEventListener('click', function(){
     const item = document.getElementById('itemInput').value;
     const newli= document.createElement('li');
     newli.textContent = item;

     newli.addEventListener('click',function(){
        console.log('item clicked');
        newli.classList.toggle('done');
     });

     document.getElementById('todoList').appendChild(newli);
  });
```

JQuery Code:

_____

_____

Activity 5: Style the "Focus on Me" Button

Objective: Use class toggling and transitions for styling on hover.

Task:·

Add a `<span>` with text "focus on me". **- given**

Style it using CSS.

On `mouseenter`, add a class that animates the button.

On `mouseleave`, remove the class.

.

Add this CSS style:

```
.focus-button {
  position: fixed;
  bottom: 20px;
  left: 20px;
  padding: 10px 20px;
  background-color: #007BFF;
  color: white;
  border-radius: 20px;
  transition: all 0.3s ease;
}
.floating {
  transform: translateY(-10px) scale(1.1);
  box-shadow: 0 10px 20px rgba(0, 0, 0, 0.2);
}
```

Javascript Code:

_____

_____

JQuery Code:

_____

_____